# Button

DETECT | RASPBERRY PI | INPUT

## What does it do?

A button lets someone tell your program to do something the moment they press.

Inside are two metal contacts. Pressing pushes them together so electricity can flow, and letting go springs them apart again.

Your projects can use it to start something, stop something, or count how many times it has been pressed.

## Uses



### Pedestrian crossing

**DETECT** Button and traffic phase → **DECIDE** Is it safe to cross? → **DO** Change the lights



### Do not press this

**DETECT** Button is pressed → **DECIDE** Is the trap armed? → **DO** Play a silly sound



**Cap**
Press here

**Legs**
Four, joined in pairs

**Gap**
Straddle the channel



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import Button

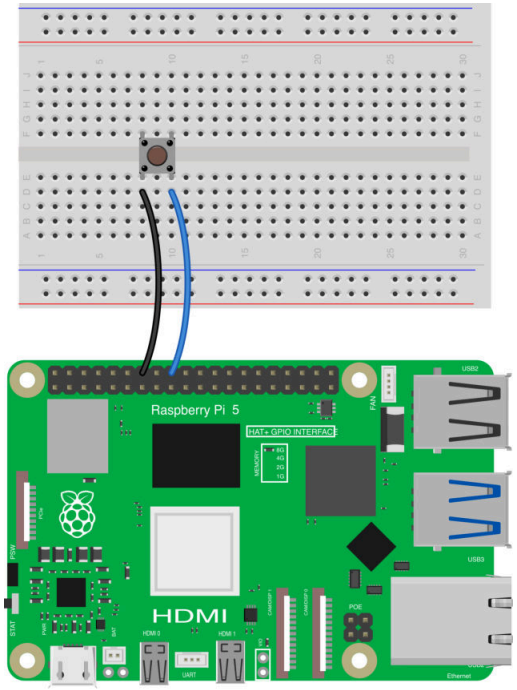
button = Button(24)

pressed = button.is_pressed
```

`pressed` is True or False. No resistor is needed — gpiozero switches on the pin's own pull-up. Also `button.when_pressed` and `button.when_released`.

# Wiring

**RASPBERRY PI 5**




**GPIO 24** (pin 18) → button → **GND** (pin 14)

# Capacitive touch

DETECT | RASPBERRY PI | INPUT

## What does it do?

A capacitive touch sensor notices when a finger comes close to its metal pad.

It detects the tiny change in electric charge caused by your body, so no moving parts are needed.

Use it anywhere a project needs a hidden, wipe-clean or feather-light button.

## Uses



### Touch lamp

DETECT: Finger touches the pad → DECIDE: Is the lamp already on? → DO: Toggle the light



### Banana piano

DETECT: A banana is touched → DECIDE: Which note should play? → DO: Play the note




**Touch pad**
The large metal spiral

**SIG**
Digital output

**VCC and GND**
3.3 V power



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import DigitalInputDevice

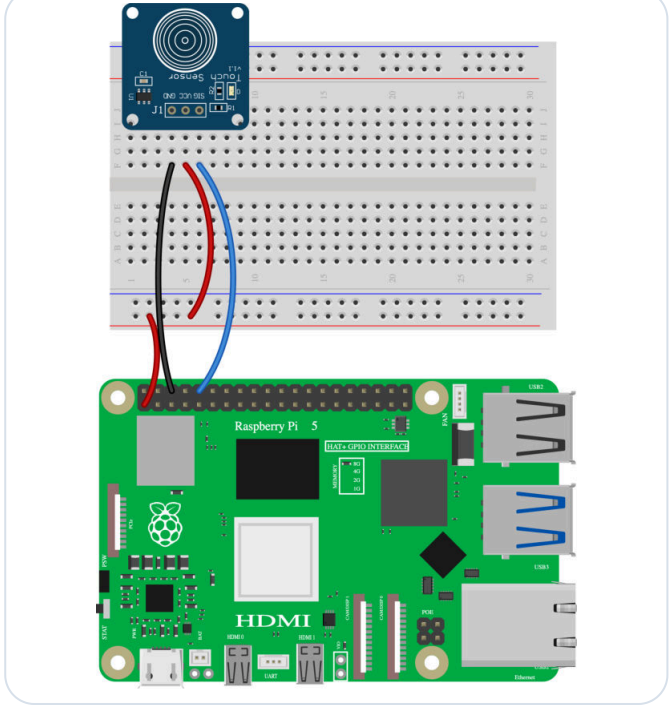
touch = DigitalInputDevice(15, pull_up=False)

touched = touch.is_active
```

The TTP223 output goes high while touched. `touched` is True or False.

# Wiring

**RASPBERRY PI 5**




**GND** (pin 6) → GND, **3V3** (pin 1) → VCC, **GPIO 15** (pin 10) → SIG

# LDR

DETECT | RASPBERRY PI | LIGHT

## What does it do?

An LDR is a sensor that measures how bright or dark it is.

LDR stands for light-dependent resistor. Light shining on it lets electricity through more easily, so your code can tell light from dark.

Your projects can use it to react when a lamp comes on, when night falls, or when a shadow passes over.

## Uses



### Street light

**DETECT** How dark it is → **DECIDE** Is it darker than dusk? → **DO** Switch the lamp on



### Biscuit tin alarm

**DETECT** Light inside the tin → **DECIDE** Is the lid open? → **DO** Sound the alarm



**Window** Light lands here

**Track** Zigzag sensor

**Legs** Either way round



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import LightSensor

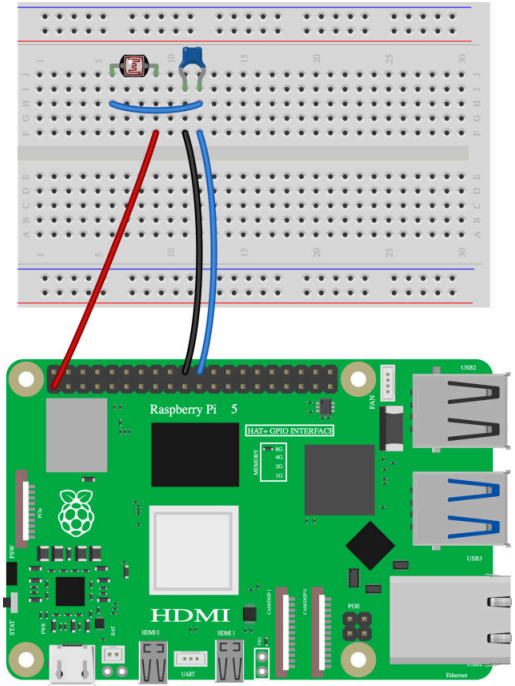
ldr = LightSensor(25)

light = ldr.value
```

`light` runs from 0 in the dark to 1 in bright light. Also `ldr.light_detected` and `ldr.when_light`.

# Wiring

**RASPBERRY PI 5**




**3V3** (pin 1) → LDR → **GPIO 25** (pin 22) → 1 µF → **GND** (pin 20)

# Line sensor

DETECT | RASPBERRY PI | INPUT

## What does it do?

A line sensor tells light and dark surfaces apart underneath a moving project.

An infrared LED shines at the floor and a detector measures how much light bounces back.

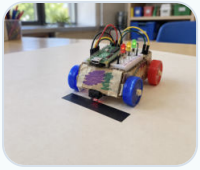
Adjust the small potentiometer until its digital output changes reliably at the line edge.

## Uses



### Warehouse guide-line robot

**DETECT** Read guide-line position → **DECIDE** Is the vehicle drifting? → **DO** Correct the steering



### Cardboard lap-counting car

**DETECT** Cross dark finish strip → **DECIDE** Is this a new lap? → **DO** Increase the lap count




**Infrared pair**
Faces the floor

**Adjuster**
Sets the threshold

**S**
Digital signal



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import LineSensor

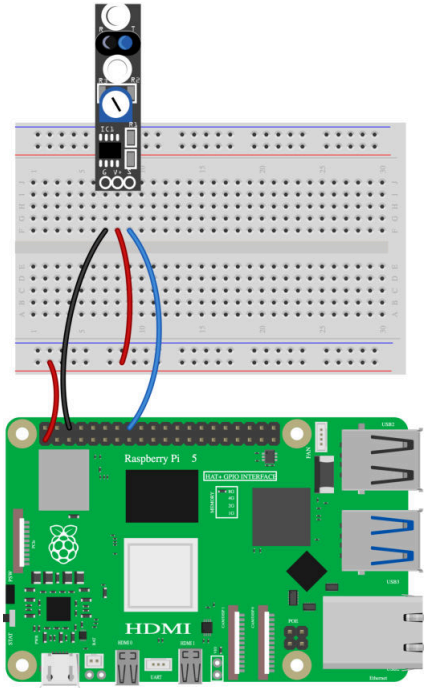
line = LineSensor(23)

on_line = line.is_active
```

> Turn the blue adjuster until the module's output changes cleanly at the edge of the line.

# Wiring

**RASPBERRY PI 5**




**GND** (pin 6) → G, **3V3** (pin 1) → V+, **GPIO 23** (pin 16) → S

# Distance sensor

DETECT | RASPBERRY PI | DISTANCE

## What does it do?

An HC-SR04P distance sensor measures how far away something is without touching it.

It works like a bat. One side sends out a click too high for you to hear, the other side listens for the echo, and the time it takes gives the distance.

Your projects can use it to react as something comes closer, or to measure a gap that keeps changing.

## Uses



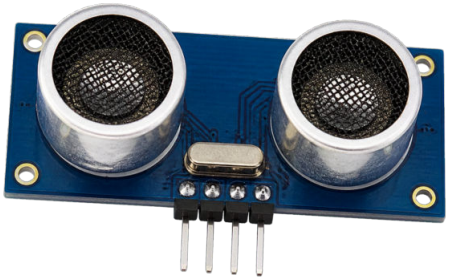
### Parking sensor

DETECT: Gap to the wall → DECIDE: Is it closer than 30 cm? → DO: Beep faster



### Air harp

DETECT: How far your hand is → DECIDE: Which note is that? → DO: Play the note



| VCC | Trig | Echo | GND |
|---|---|---|---|
| Power, 3V3 | Sends a click | Times the return | Ground |



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import DistanceSensor

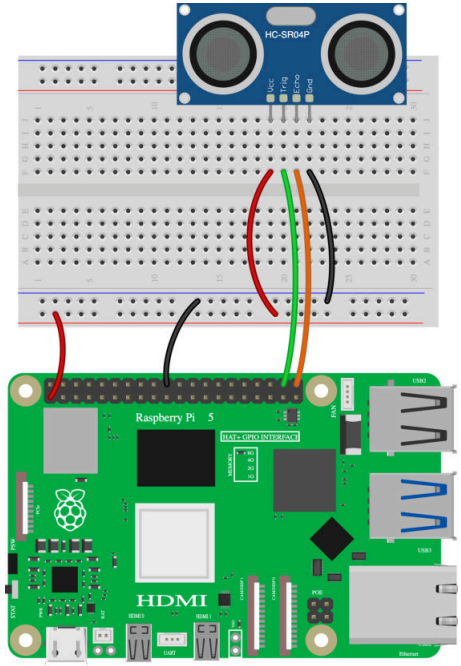
sensor = DistanceSensor(
    echo=21, trigger=20
)

distance = sensor.distance
```

`distance` is in metres and stops at 1. Pass `max_distance=2` to see further, or read `sensor.value` for 0 to 1. Use an **HC-SR04P**, powered from **3V3**.

# Wiring

**RASPBERRY PI 5**




HC-SR04P · **3V3** (pin 1) to + rail · Trig **GPIO 20** (pin 38) · Echo **GPIO 21** (pin 40) · **GND** (pin 20) to − rail

# PIR

DETECT | RASPBERRY PI | MOTION

## What does it do?

A PIR spots movement nearby, such as somebody walking past.

PIR stands for passive infrared. It watches the warmth that bodies give off and reports when that warmth moves, so it notices people and animals rather than furniture.

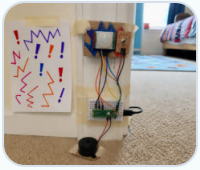
Your projects can use it to wake something up the moment somebody arrives, and go quiet again when they leave.

## Uses



### Burglar alarm

**DETECT** Movement in the room → **DECIDE** Should the room be empty? → **DO** Raise the alarm



### Sibling alarm

**DETECT** Movement at the door → **DECIDE** Is privacy mode armed? → **DO** Sound the warning



| Dome | VCC | OUT | GND |
|---|---|---|---|
| Wide-angle lens | Power, 5V | High on movement | Ground |



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import MotionSensor

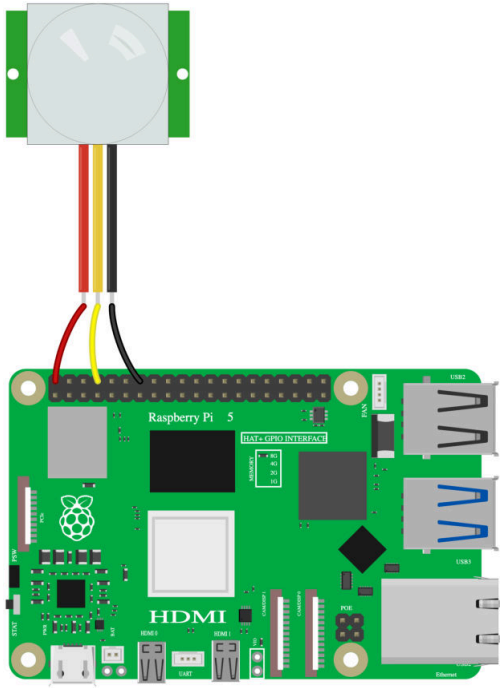
pir = MotionSensor(14)

moved = pir.motion_detected
```

`moved` is True or False. Also `pir.when_motion` and `pir.when_no_motion` to run a function the moment it changes. Most boards carry two dials that set how sensitive it is and how long OUT stays high.

# Wiring

**RASPBERRY PI 5**




**5V** (pin 2) · OUT **GPIO 14** (pin 8) · **GND** (pin 14)

# AM312 PIR

DETECT | RASPBERRY PI | MOTION | MINI PIR

## What does it do?

An **AM312** is a tiny PIR sensor that notices warm bodies moving nearby.

PIR means passive infrared. Its faceted lens divides the view into zones, so a person or animal moving between them makes the output switch on.

The complete AM312 module runs from 3V3 and uses very little power. Its sensitivity and roughly two-second output pulse are fixed.

## Uses



### Wardrobe light

**DETECT** Movement in the wardrobe → **DECIDE** Is someone reaching in? → **DO** Turn on the light



### Dragon treasure alarm

**DETECT** Movement near the hoard → **DECIDE** Is someone sneaking closer? → **DO** Make the dragon roar



| Lens | VCC | OUT | GND |
|---|---|---|---|
| Splits the view into zones | Power from 3V3 | High on movement | Ground |



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import MotionSensor

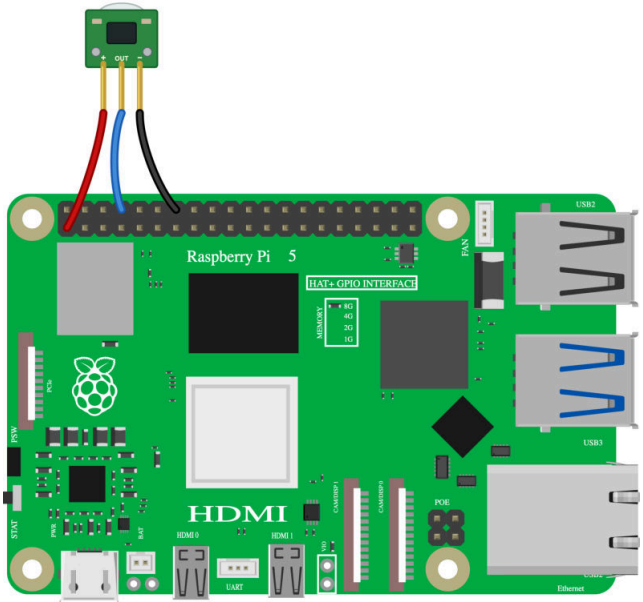
pir = MotionSensor(14)

moved = pir.motion_detected
```

`moved` is True or False. Use `pir.when_motion` and `pir.when_no_motion` to run functions when it changes. The AM312 has no sensitivity or delay dials; OUT normally stays high for about two seconds after movement.

# Wiring

**RASPBERRY PI 5**




**VCC** → 3V3 (pin 1); **OUT** → GPIO14 (pin 8); **GND** → GND (pin 14).

# Potentiometer

DETECT | RASPBERRY PI | INPUT

## What does it do?

A potentiometer lets your code read how far a knob has been turned.

This small version is called a trimpot. Turning the blue knob moves a contact called the wiper, changing the voltage your board reads.

Use it to choose a speed, a brightness or a sound. Turn gently and stop at the ends. The marking 103 means 10 kΩ.

## Uses



### Security-light adjustment

DETECT: Read sensitivity setting → DECIDE: Is movement strong enough? → DO: Switch the light on



### Alien voice tuner

DETECT: Knob position → DECIDE: Should it rumble or squeak? → DO: Rumble or squeak




| Knob | Outer pair | Wiper |
|---|---|---|
| Turn to adjust | Pins 1 and 3 | Pin 2, offset |

# Code

## Raspberry Pi

```
from gpiozero import MCP3008

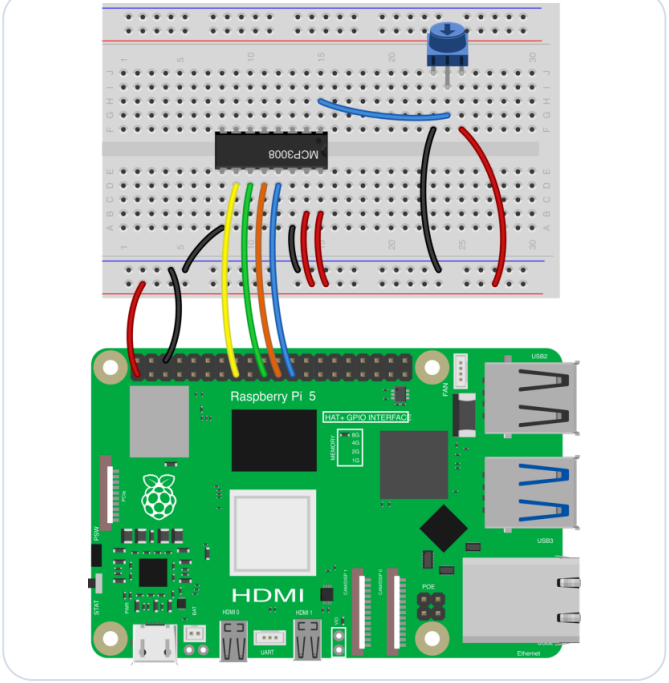
pot = MCP3008(0, select_pin=22)

setting = pot.value
```

`setting` runs from 0 to 1. Read `pot.value` again after turning; `pot.voltage` gives volts. The Pi needs the MCP3008 converter shown. Power the trimpot and converter from **3V3**.

# Wiring

**PI 5 + MCP3008**




Wiper → **CH0**. CLK / DIN / DOUT / CS → **GPIO 11 / 10 / 9 / 22** (pins 23 / 19 / 21 / 15). **3V3** (pin 1), **GND** (pin 6). Chip notch faces right.

# Temperature sensor

DETECT | RASPBERRY PI | INPUT | ANALOGUE

## What does it do?

A TMP36 turns temperature into a changing voltage that a program can measure.

Its output rises by 10 millivolts for every degree Celsius, with 0.5 V representing 0 °C.

The Pico reads that voltage directly; the Raspberry Pi uses an MCP3008 analogue-to-digital converter.

## Uses



### Refrigerator cooling

**DETECT** Measure fridge temperature → **DECIDE** Is it too warm? → **DO** Increase the cooling



### Soup drama meter

**DETECT** Measure bowl surface → **DECIDE** Which drama level fits? → **DO** Move the drama pointer




| Flat face | +Vs | Vout | GND |
|---|---|---|---|
| Use it to identify the pins | Left pin | Centre pin | Right pin |



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import MCP3008

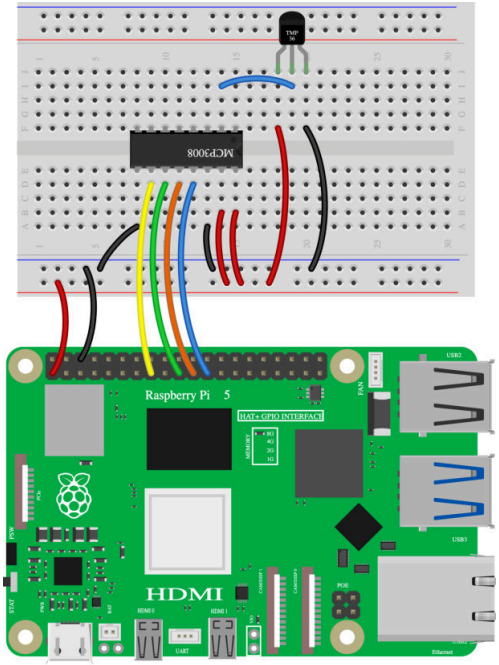
sensor = MCP3008(channel=1, select_pin=22)

temperature = (sensor.voltage - 0.5) * 100
```

Read the TMP36 with its flat face towards you; the pins are +Vs, Vout, GND from left to right.

# Wiring

**RASPBERRY PI 5**




TMP36 **Vout** → MCP3008 **CH1**. CLK / DIN / DOUT / CS → **GPIO 11 / 10 / 9 / 22** (pins 23 / 19 / 21 / 15). **3V3** (pin 1), **GND** (pin 6).

# Temperature probe

DETECT | RASPBERRY PI | DS18B20 | DIGITAL

## What does it do?

A **DS18B20** measures temperature and sends a digital reading in degrees Celsius.

This probe has a sealed metal tip on a cable with three wires. Check the supplier's pinout; wire colours can vary.

Only the sealed probe goes in water. Keep the board and all wire connections dry.

## Uses



### Aquarium temperature alarm

**DETECT** Measure tank temperature → **DECIDE** Is it too cold? → **DO** Show a warning



### Penguin bath alarm

**DETECT** Measure bath temperature → **DECIDE** Is it too warm? → **DO** Light a warning LED



| VDD | DQ | GND | 4.7 kΩ |
|---|---|---|---|
| Power from 3V3 | Digital data | Ground | Resistor from DQ to 3V3 |



Raspberry Pi Foundation

# Code

## Raspberry Pi

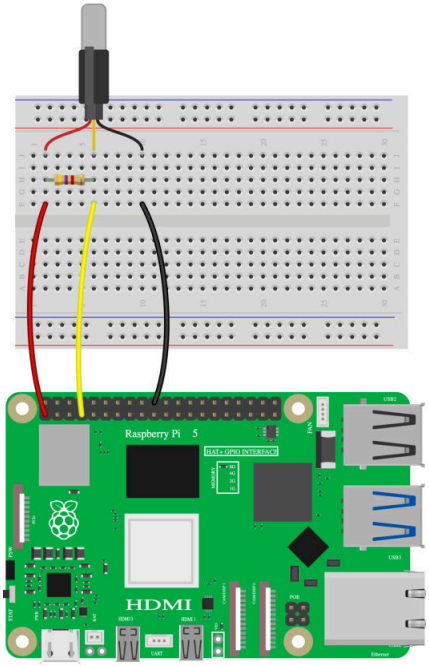
```
from w1thermsensor import W1ThermSensor

sensor = W1ThermSensor()
temperature = sensor.get_temperature()
print(temperature)
```

**Setup:** enable 1-Wire in `raspi-config`, reboot, then run `sudo apt install python3-w1thermsensor`.

# Wiring

**RASPBERRY PI 5**




**VDD** → 3V3 (pin 1); **DQ** → GPIO4 (pin 7); **GND** → GND (pin 20). **4.7 kΩ** between DQ and 3V3.

# Tilt switch

DETECT | RASPBERRY PI | INPUT

## What does it do?

A tilt switch changes state when it tips past a particular angle.

While it stands upright, a tiny metal ball rests on two contacts and completes the circuit. Tip it over and the ball rolls away, breaking the circuit.

It can detect that an object has been picked up, tipped over or opened.

## Uses



### Iron safety shut-off

DETECT: Iron tips sideways → DECIDE: Has the delay elapsed? → DO: Switch the heater off



### Grumpy pencil case

DETECT: Case is tipped → DECIDE: Has it stayed tipped? → DO: Make a grumpy noise



**Metal can**
Contains the rolling ball

**Legs**
No polarity

**Angle**
Changes the contact



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import Button

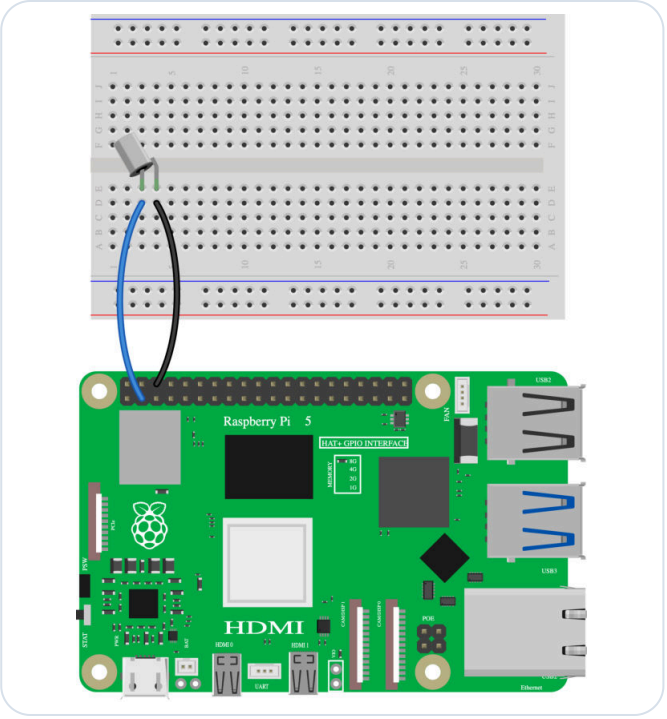
tilt = Button(2)

tilted = not tilt.is_pressed
```

> The reading may flicker while the ball rolls. Check it more than once before treating it as a definite tilt.

# Wiring

**RASPBERRY PI 5**




**GPIO 2** (pin 3) → tilt switch → **GND** (pin 6)

# Toggle switch

DETECT | RASPBERRY PI | INPUT

## What does it do?

A toggle switch stays in the position you move it to, unlike a push button.

Its centre COM leg connects to one outside leg in each position.

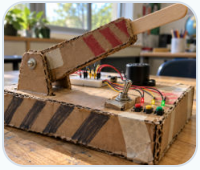
It is useful when a project needs a lasting on/off choice or two modes.

## Uses



### Auto/manual selector

**DETECT** Read selector position → **DECIDE** Is automatic mode selected? → **DO** Run the selected mode



### Serious business lever

**DETECT** Lever and check buttons → **DECIDE** Are all checks complete? → **DO** Start unnecessary sirens



| Lever | COM | L1 and L2 |
|---|---|---|
| Stays where it is moved | Centre pin | The two choices |

# Code

## Raspberry Pi

```
from gpiozero import Button

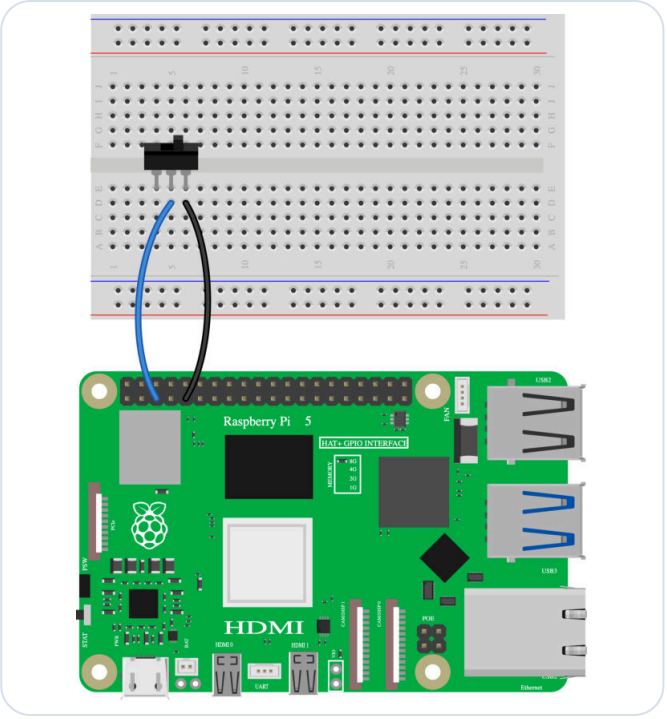
mode = Button(3)

mode_two = mode.is_pressed
```

Connect COM and one outside pin. The unused outside pin can be left unconnected.

# Wiring

**RASPBERRY PI 5**




**GPIO 3** (pin 5) → COM; right-hand outside pin → **GND** (pin 9)

# Pi Camera

DETECT | RASPBERRY PI | IMAGES | VIDEO

## What does it do?

A camera turns light into digital images. Connect a Raspberry Pi Camera Module to a Raspberry Pi computer with a flat ribbon cable.

Use Python to take a photograph or record a video. Your program can also examine images and decide what to do next.

## Uses



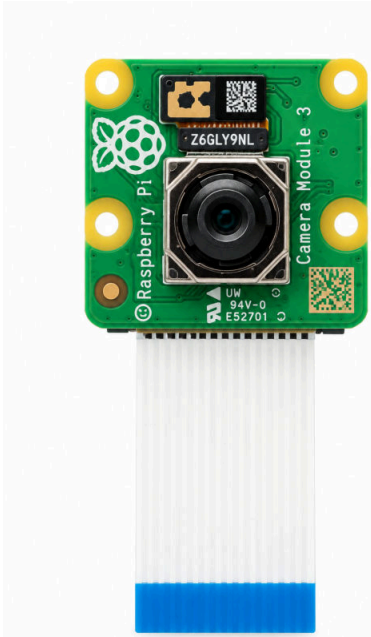
### Video doorbell

**DETECT** Camera sees movement. → **DECIDE** Is a person there? → **DO** Save a video clip.



### Monster photo booth

**DETECT** Button starts countdown. → **DECIDE** Has time run out? → **DO** Take a silly photo.




**Lens** Focuses light

**Sensor** Records the image

**Ribbon** Connects to the Pi

# Code

## Take photo

```
from picamera2 import Picamera2

camera = Picamera2()
camera.start_and_capture_file(
    "photo.jpg", delay=2,
    show_preview=False)
camera.close()
```

## Take video

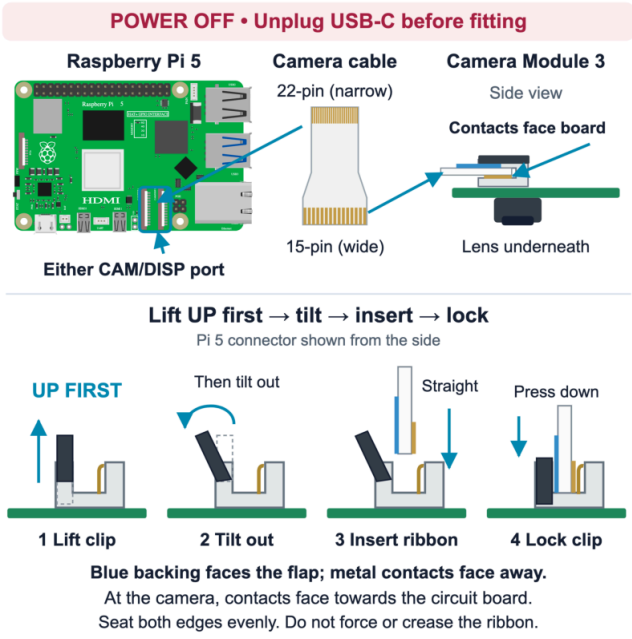
```
from picamera2 import Picamera2

camera = Picamera2()
camera.start_and_record_video(
    "video.mp4", duration=5)
camera.close()
```

Run each example separately on **Raspberry Pi OS** with **Picamera2** installed. Files save in the current folder. The video is five seconds long, without sound.

# Wiring

**RASPBERRY PI 5 + CAMERA MODULE 3**




Use a **Standard–Mini camera cable** (22-pin to 15-pin). The wide cable supplied with the camera fits a Pi 4, not a Pi 5.

# LED

DO | RASPBERRY PI | LIGHT

## What does it do?

An LED is a tiny light that your code can switch on, switch off, or blink.

LED stands for light-emitting diode. It glows when electricity flows the right way through it, so the long leg and the short leg matter.

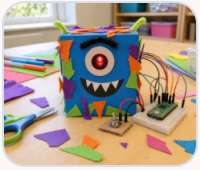
Your projects can use LEDs to show a warning, celebrate something, or tell people what is happening.

## Uses



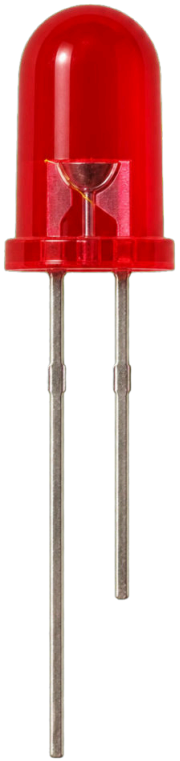
### Car brake light

**DETECT** Brake control signal → **DECIDE** Is braking requested? → **DO** Light the red LEDs



### One-eyed monster box

**DETECT** Box lid is opened → **DECIDE** Is guard mode armed? → **DO** Glow the monster eye



**Anode**
Long leg, to GPIO

**Cathode**
Short leg, to GND

**Lens**
Glows here



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import LED

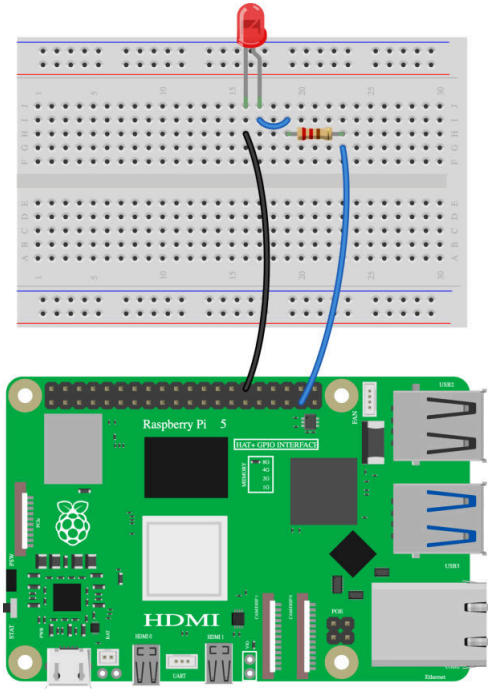
led = LED(26)

led.on()
```

Also `led.off()`, `led.toggle()` and `led.blink()`. Give `blink()` an `on_time` and an `off_time` to set the flash speed.

# Wiring

**RASPBERRY PI 5**




**GPIO 26** (pin 37) → 220 Ω → LED → **GND** (pin 30)

# RGB LED

DO | RASPBERRY PI | OUTPUT | LIGHT

## What does it do?

An RGB LED puts red, green and blue lights inside one clear package.

Varying the brightness of the three colours mixes millions of possible colours.

A common-cathode LED has one shared negative leg and needs one resistor for each colour.

## Uses



### Wi-Fi router status

**DETECT** Read connection status → **DECIDE** Which status colour is needed? → **DO** Mix the LED colour



### Mood crystal

**DETECT** Read the mood dial → **DECIDE** What is the crystal's mood? → **DO** Glow that colour



**Clear lens**
Contains three LEDs

**Long leg**
Common cathode

**R, G and B legs**
One 220 ohm resistor each



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import RGBLED

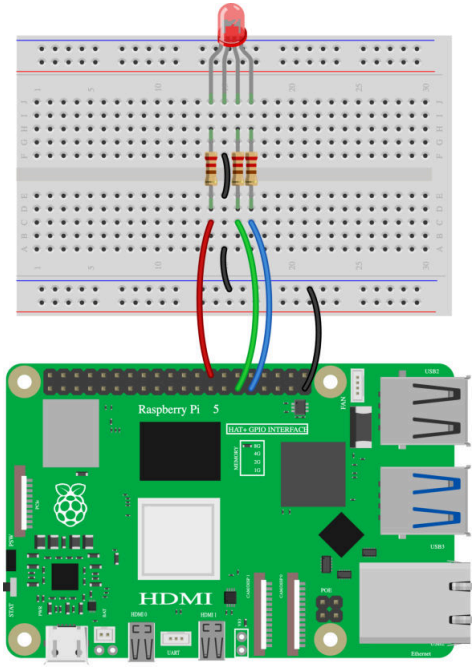
light = RGBLED(red=7, green=5, blue=6)

light.color = (1, 0, 1)
```

Colour values run from 0 to 1. This example makes magenta.

# Wiring

**RASPBERRY PI 5**




R → **GPIO7** (pin 26), G → **GPIO5** (pin 29), B → **GPIO6** (pin 31); common → **GND** (pin 39)

# NeoPixels

DO | RASPBERRY PI | OUTPUT | LIGHT

## What does it do?

A NeoPixel is a full-colour LED with a tiny controller built into it.

One data wire carries colour instructions, and each pixel passes the remaining instructions onwards.

The 3.3 V-compatible JST breakout shown here can connect directly to a Pi or Pico.

## Uses



### Gaming PC lighting

**DETECT** Read lighting selection → **DECIDE** Which colour should each pixel be? → **DO** Light individual pixels



### Disco shoelace meter

**DETECT** Read shoe movement → **DECIDE** Which light pattern should run? → **DO** Animate the pixels




**Pixel** RGB LED and controller

**IN** Data from the board

**OUT** Data to another pixel

**VDD and GND** 3.3 V power



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
import board
import neopixel

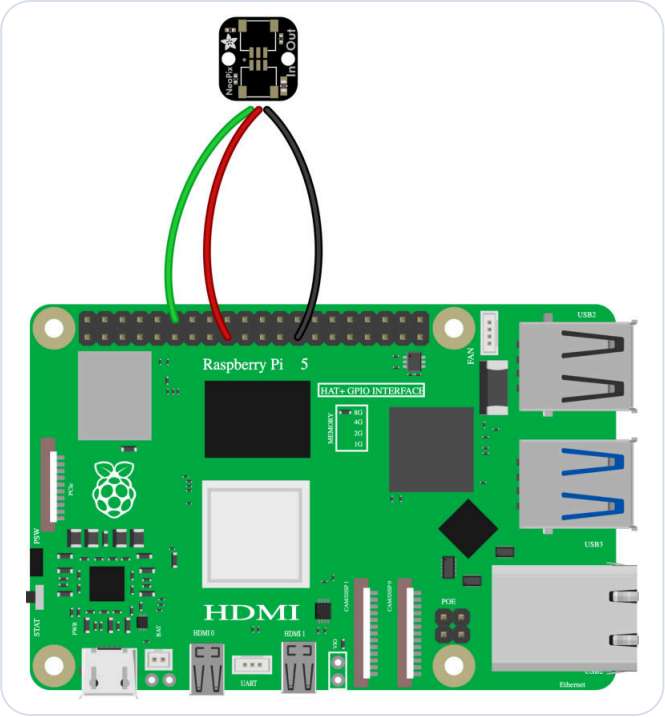
pixels = neopixel.NeoPixel(board.D18, 1)

pixels[0] = (255, 0, 80)
```

This wiring is for the pictured 3.3 V-compatible breakout. Generic 5 V strips usually need a separate supply and a logic-level shifter.

# Wiring

**RASPBERRY PI 5**




**GPIO18** (pin 12) → IN, **3V3** (pin 17) → VDD, **GND** (pin 25) → GND

# Active buzzer

DO | RASPBERRY PI | SOUND

## What does it do?

An active buzzer makes one built-in tone. Switch it on and off for warning beeps or rhythms; its pitch stays the same.

**Recognise it:** check the packet for **active** or **built-in oscillator**. Shape, stickers and + marks do not prove the type.

Use a labelled **3.3 V GPIO-compatible** buzzer.

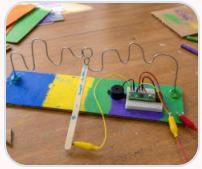
## Uses



### Smoke alarm

One warning tone is enough; no changing notes needed.

**DETECT** Measure smoke level → **DECIDE** Is smoke above the limit? → **DO** Repeat one warning tone



### Buzz wire game

Choose active: one fixed buzz clearly signals failure.

**DETECT** Loop touches wire → **DECIDE** Is this the third strike? → **DO** Buzz at one pitch



**+ leg**
Marked +, to GPIO

**– leg**
Other leg, to GND

**Vent**
Sound comes out



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import Buzzer

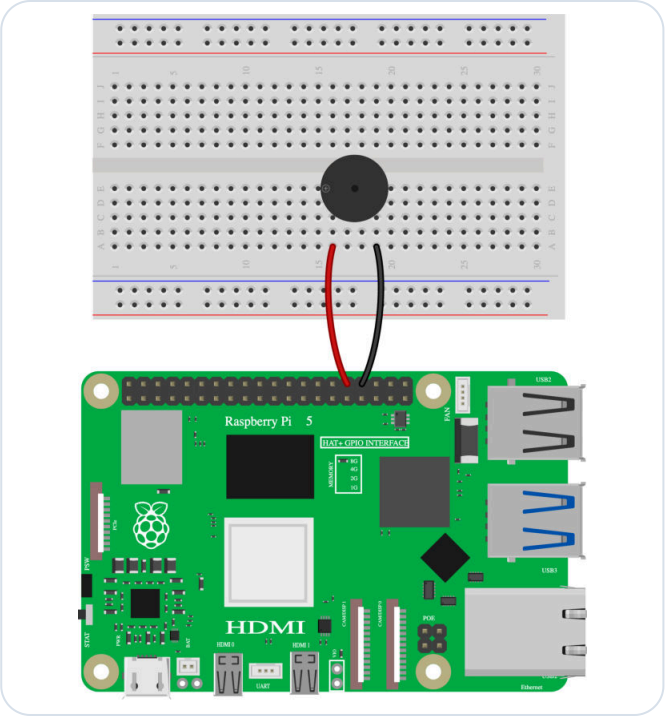
buzzer = Buzzer(12)

buzzer.on()
```

Also `buzzer.off()`, `buzzer.toggle()` and `buzzer.beep()`. Give `beep()` an `on_time` and an `off_time` to set the rhythm. Faster beeps do not mean a higher pitch.

# Wiring

**RASPBERRY PI 5**




**GPIO 12** (pin 32) → **+** buzzer **−** → **GND** (pin 34)

# Passive buzzer

DO | RASPBERRY PI | SOUND | PWM

## What does it do?

A passive buzzer needs fast pulses (PWM) to make sound. Changing their frequency changes its pitch: low or high notes.

**Recognise it:** check for **passive**, **external drive** or **no oscillator** on the packet. Both types can look identical; + only marks polarity.

Use a small **3.3 V GPIO-compatible piezo** sounder.

## Uses



### Washing-machine chime

PWM plays tunes; active buzzers have only one pitch.

**DETECT** Read cycle status → **DECIDE** Has the wash finished? → **DO** Play a short tune



### Robot voice box

Change pitch for happy chirps or grumpy growls.

**DETECT** Read mood button → **DECIDE** Is happy pressed? → **DO** Chirp high or low



**+ leg**
Marked +, to GPIO

**– leg**
Other leg, to GND

**Vent**
Sound comes out



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import TonalBuzzer

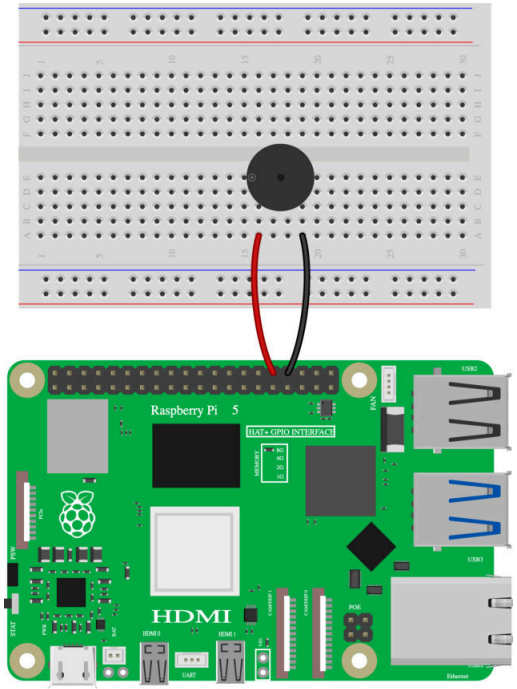
buzzer = TonalBuzzer(12)

buzzer.play("A5")
```

Try `buzzer.play("A4")`; silence with `buzzer.stop()`. Higher frequency means higher pitch. Beep speed is separate; both types can make rhythms.

# Wiring

**RASPBERRY PI 5**




**GPIO 12** (pin 32) → **+** buzzer **−** → **GND** (pin 34)

# Speaker

DO | RASPBERRY PI | OUTPUT | SOUND

## What does it do?

A small passive speaker turns a changing electrical signal into sound.

Current through its coil moves a cone backwards and forwards, pushing the air.

The 220 ohm resistor shown here protects the GPIO; it also keeps this simple circuit fairly quiet.

## Uses



### Door chime

**DETECT** Button and chime setting → **DECIDE** Which chime should play? → **DO** Play the notes



### Tiny fanfare machine

**DETECT** Count achievement-button presses → **DECIDE** Has the target been reached? → **DO** Play triumphant notes



**Cone**
Pushes the air

**Coil and magnet**
Create movement

**Terminals**
Connect through 220 ohms



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import TonalBuzzer

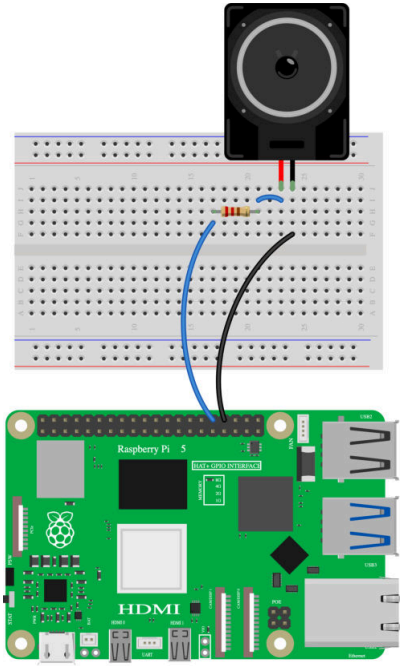
speaker = TonalBuzzer(12)

speaker.play("E4")
```

For a louder speaker, use a transistor or amplifier rather than removing the protective resistor.

# Wiring

**RASPBERRY PI 5**




**GPIO12** (pin 32) → 220 ohms → speaker red (+) lead; black lead → **GND** (pin 34)

# Motor driver

DO | RASPBERRY PI | OUTPUT | MOVEMENT

## What does it do?

A motor driver lets a low-power GPIO control a motor that needs more current.

The L293D contains H-bridges that can send current through a motor in either direction.

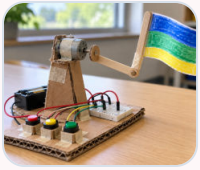
Two input pins choose forwards, backwards or stop, while the chip handles the motor current.

## Uses



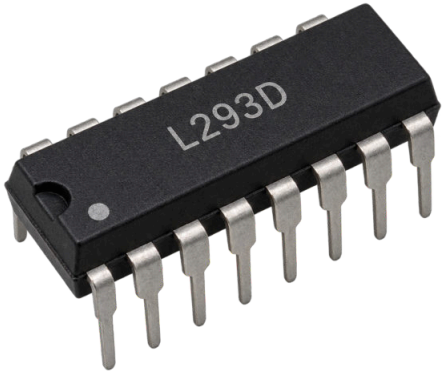
### Electric car window

**DETECT** Read window switch → **DECIDE** Can it move safely? → **DO** Drive requested direction



### Tiny flag waver

**DETECT** Count score-button presses → **DECIDE** Has the target been reached? → **DO** Wave the flag




**Notch**
Identifies pin 1

**IN1 and IN2**
Direction controls

**OUT1 and OUT2**
Connect to the motor

**Enable**
Held high in this circuit

# Code

## Raspberry Pi

```
from gpiozero import Motor

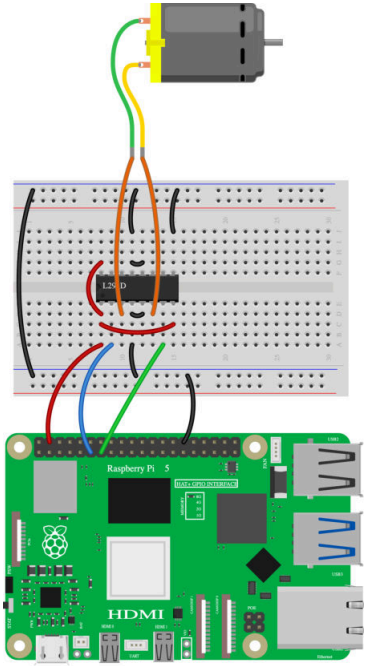
motor = Motor(forward=17, backward=27)

motor.forward()
```

This card powers one small 5 V motor from the board's 5 V rail. Use a separate motor supply for larger motors, with the grounds joined.

# Wiring

**RASPBERRY PI 5**




**GPIO17** (pin 11) → IN1, **GPIO27** (pin 13) → IN2; OUT1 and OUT2 → motor; **5V** (pin 4), **GND** (pin 30)

# Servo motor

DO | RASPBERRY PI | OUTPUT | MOVEMENT

## What does it do?

A positional servo moves its arm, called a horn, to a chosen position and holds it there.

Short pulses on its signal wire set the position. The motor and position control are built into the case.

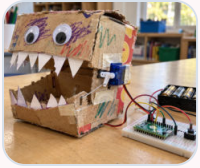
Use it for steering, pointing or opening a flap. A continuous-rotation servo controls speed instead; it cannot use this example to choose an angle.

## Uses



### Radio-controlled steering

**DETECT** Read steering command → **DECIDE** Which way should it turn? → **DO** Set the steering position



### Cardboard chomping monster

**DETECT** Read feeding button → **DECIDE** Is the monster hungry? → **DO** Open its cardboard jaw



| Horn | Signal | Power | Ground |
|---|---|---|---|
| Attach a light moving part | Usually orange, yellow or white | Usually red; about 5 V | Brown or black; join grounds |



Raspberry Pi Foundation

# Code

## Raspberry Pi

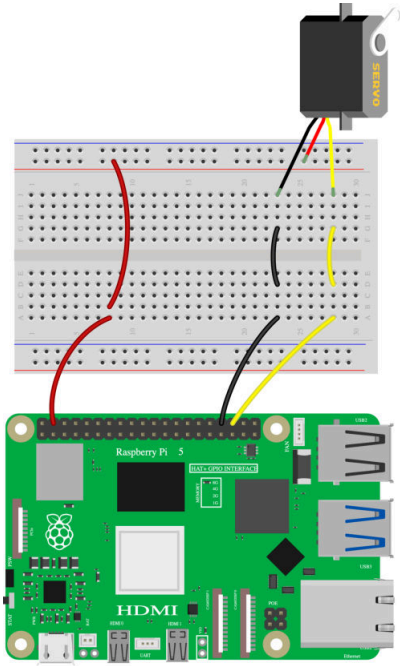
```
from gpiozero import Servo
from time import sleep

servo = Servo(16)
servo.mid()
sleep(2)
```

Test unloaded: the horn centres. Try min() or max() instead of mid(). Travel varies; stop if it binds or buzzes. Check lead order.

# Wiring

**RASPBERRY PI 5**




**GPIO16** (pin 36) → signal; **5V** (pin 4) → top + rail → servo red; **GND** (pin 34) → servo −. Small **SG90-size** servo only: the Pi powers it.

# Stepper motor

DO | RASPBERRY PI | OUTPUT | MOVEMENT

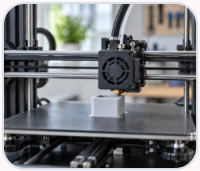
## What does it do?

A stepper motor turns in small, repeatable steps instead of spinning freely.

Energising its coils in sequence moves the shaft one step at a time and holds its position.

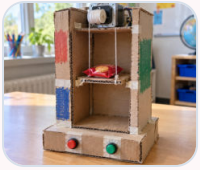
The ULN2003 board supplies the coil current for the common 28BYJ-48 motor shown here.

## Uses



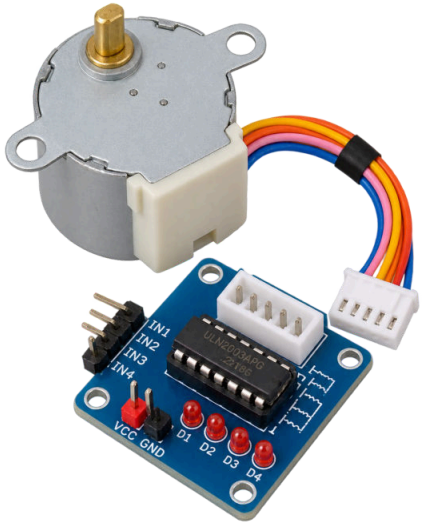
### 3D printer positioning

**DETECT** Read next print position → **DECIDE** How many steps are needed? → **DO** Move the print head



### Snack lift

**DETECT** Snack request arrives → **DECIDE** Which floor needs it? → **DO** Step lift into place




**Output shaft** Moves in measured steps

**Five-wire plug** Connects to the driver

**IN1 to IN4** Coil sequence

**5V and GND** Driver power



Raspberry Pi Foundation

# Code

## Raspberry Pi

```
from gpiozero import OutputDevice
from time import sleep

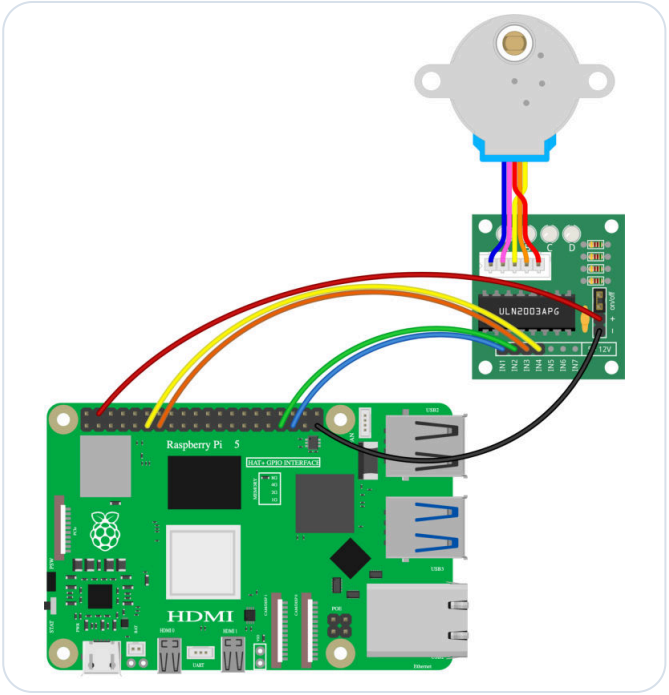
coils = [OutputDevice(pin) for pin in (19, 13, 27, 17)]
sequence = ((1, 0, 0, 0), (0, 1, 0, 0),
            (0, 0, 1, 0), (0, 0, 0, 1))

for pattern in sequence * 512:
    for coil, value in zip(coils, pattern):
        coil.value = value
    sleep(0.002)
```

`sequence * 512` runs about one full turn. Reverse the sequence to turn the other way.

# Wiring

**RASPBERRY PI 5**




IN1-IN4 → GPIO **19, 13, 27, 17** (pins 35, 33, 13, 11); driver → **5V** (pin 4) and **GND** (pin 39)

# Robot / rover

DO | RASPBERRY PI | OUTPUT | MOVEMENT

## What does it do?

A two-wheel rover uses a separate geared motor for each wheel. A caster or smooth skid supports the other end.

Drive both wheels forwards to move; change their speeds or directions to turn. A motor driver handles their power.

The Robot class joins the two motors into one object. This example uses a breadboard-mounted L293D chip and a separate motor battery pack.

## Uses



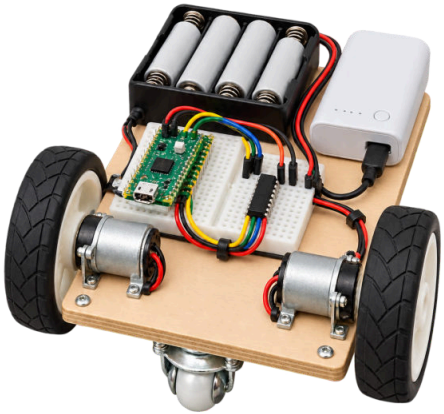
### Robot vacuum cleaner

**DETECT** Sense a nearby obstacle → **DECIDE** Is the path clear? → **DO** Drive on or turn away



### Sock delivery turtle

**DETECT** Read delivery button → **DECIDE** Is a delivery requested? → **DO** Drive the sock parcel



**Drive wheels**
One geared motor per side

**Motor driver**
L293D, a 16-pin breadboard chip

**Caster or skid**
Supports the chassis

**Power**
Separate motor and USB supplies



Raspberry Pi Foundation

# Code

## Raspberry Pi

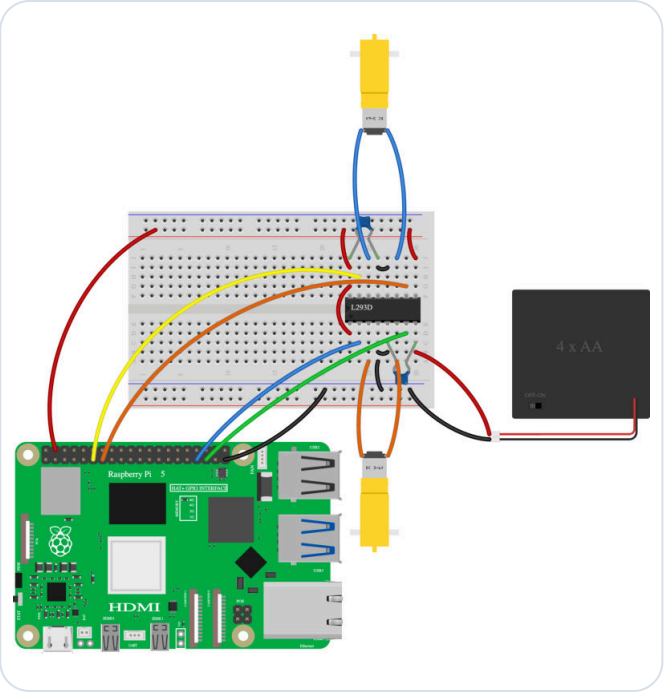
```
from gpiozero import Motor, Robot
from time import sleep

robot = Robot(left=Motor(13, 19),
              right=Motor(17, 27))
try:
    robot.forward()
    sleep(1)
finally:
    robot.stop()
```

Test with wheels lifted. Reverse a motor by swapping its leads with power off. Try left(), right(), backward() and stop().

# Wiring

**RASPBERRY PI 5**




**GPIO13/19** (Pi pins 33/35) → L293D pins 2/7; **GPIO17/27** (Pi pins 11/13) → 15/10. **5 V** (Pi pin 4) → 1, 9, 16. **GND** (Pi pin 39) → 4, 5, 12, 13.